Software Engineering 2
Lösungen der Übung JavaScript, Nachladen von Daten im HTML (AJAX)

Dr. F.-K. Koschnick, Sybit GmbH

Copyright © 2024 Dr. F. K. Koschnick, Sybit GmbH. All rights reserved.

Aufgabe (Story): Nachladen von Daten im HTML

Übung 4

Als Nutzer der Webseite möchte ich möglichst schnell ohne ein vollständiges Neuladen der Seite, Daten über Tiere und deren Maximalgeschwindigkeiten angezeigt bekommen, um ein möglichst angenehmes Nutzungserlebnis der Seite zu haben.

Verifiziere, dass
- das Layout von Bootstrap verwendet wird, so dass die nachgeladene Tabelle responsiv dargestellt wird
- es einen Button gibt, der das Nachladen der Daten auslöst
- der Webdienst (REST-Service) "http://localhost:8080/webService/api/velocitiespagination?start=..." angesprochen wird (dazu muss der Webservice in der Entwicklungsumgebung gestartet sein)
- der Parameter "start" mit 0 beginnt
- der Parameter "start" mit dem Button "next" um 5 erhöht wird
- der Parameter "start" mit dem Butten "prev" um 5 erniedrigt wird
- der Parameter niemals negativ wird und nicht weiter hochgezählt wird, wenn keine Daten mehr kommen
- das Javascript, dass die AJAX-Calls macht und den Parameter "start" steuert in einer externen JS-Datei eingebunden ist
- das Layout der Seite den angehängten Bildern (vor und nach dem Nachladen) entspricht

Technische Aspekte:
- es wird AJAX verwendet. Dieses darf mit oder ohne jQuery genutzt werden
- Bootstrap wird so im Head des HTMLs eingebunden:
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">

Für den body wird die Styleklasse "text-center" benutzt, also: `<body class="text-center">`
Für den button wird die Styleklasse "btn" und "btn-primary btn-lg" benutzt, also: `<button class="btn btn-primary btn-lg" ....`
Für die Tabelle wird die Styleklasse "table" und "table-striped" verwenden, also: `<table class = "table table-striped">....` (da die Tabelle mit Javascript erzeugt wird, bitte daran denken, die Anführungsstriche beim Attribut class mit Backslash zu escapen, wenn man mit einer String-Variable arbeitet oder anstelle der Anführungsstriche einfache Hochkommas zu verwenden.)

jQuery wird so eingebunden:
<script src="https://code.jquery.com/jquery-3.6.1.min.js" integrity="sha256-/xUj+3OJU5yExlq6GSYGSHk7tPXikynS7ogEvDej/m4=" crossorigin="anonymous"></script>
Der Webservice ist im Repository Vorlesungen im Ordner webService eingecheckt. Den Ordner webService auschecken, das Verzeichnis in den Workspace von Eclipse kopieren und als Gradle-Projekt importieren. Den Server an das Projekt binden und starten.

Lösung: HTML


```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js" defer></script>
    <script src="myscript_JQueryVelocityPagination.js" defer></script>
    <title>AJAX Example</title>
</head>
<body class="text-center">
    <br/>
    <button class="btn btn-primary btn-lg" id="prev">Prev</button>
    <button class="btn btn-primary btn-lg" id="next">Next</button>
    <br/><br/>
    <table class="table table-striped table-bordered table-hover" id="myTable"></table>
</body>
</html>
```

Einbinden von Bootstrap

Einbinden von jQuery

defer bedeutet Laden des Scripts verzögert zum Schluss

Style-Klassen aus Bootstrap

In diese Tabelle werden die Daten dynamisch über JavaScript eingebaut. Im Javascript muss man mit <tbody> arbeiten, sonst wird table-striped nicht richtig formatiert.

Lösung: Analyse JSON

Übung 4

Einzelnes Objekt, bestehend aus den Properties: animal und velocity


```
[
    {
        "id": 1,
        "animal": "Gepard",
        "velocity": "122 km/h"
    },
    {
        "id": 2,
        "animal": "Windhund",
        "velocity": "110 km/h"
    },
    {
        "id": 3,
        "animal": "Gazelle",
        "velocity": "90 km/h"
    },
    {
        "id": 4,
        "animal": "Antilope",
        "velocity": "90 km/h"
    },
    {
        "id": 5,
        "animal": "Gabelbock",
        "velocity": "85 km/h"
    },
    {
        "id": 6,
        "animal": "Strauss",
        "velocity": "70 km/h"
    },
    {
        "id": 7,
        "animal": "Pferd",
        "velocity": "70 km/h"
    },
    {
        "id": 8,
        "animal": "Feldhase",
        "velocity": "70 km/h"
    },
    {
        "id": 9,
        "animal": "Baer",
        "velocity": "50 km/h"
    },
    {
        "id": 10,
        "animal": "Flusspferd",
        "velocity": "48 km/h"
    },
    {
        "id": 11,
        "animal": "Afrikan. Elefant",
        "velocity": "40 km/h"
    },
    {
        "id": 12,
        "animal": "Mensch",
        "velocity": "38 km/h"
    },
    {
        "id": 13,
        "animal": "Rennechse",
        "velocity": "29 km/h"
    },
    {
        "id": 14,
        "animal": "Riesen-Schildkroete",
        "velocity": "0,37 km/h"
    },
    {
        "id": 15,
        "animal": "Schnecke",
        "velocity": "0,027 km/h"
    }
]
```


http://localhost:8080/webService/api/velocities (Eclipse)
http://localhost:8080/api/velocities (Idea)

Als GET-Aufruf, z.B. mit Postman, oder da GET einfach im Browser öffnen.

Array von Objekten

Äquivalent ist der Aufruf der URL:
http://localhost:8080/webService/api/velocitiespagination?start=0 (Eclipse)
http://localhost:8080/api/velocitiespagination?start=0 (Idea)

Es werden dann nur die ersten 5 Datensätze ausgegeben.

Lösung: JavaScript mit jQuery (URL von Eclipse)


```
$(document).ready(function(){
    $("#prev").prop("disabled", true);

    var count = (function () {
        var counter = -5;
        return function (next){
            if( next ) {
                counter += 5;
            } else {
                counter -= 5;
            }
            return counter}
        })();

    $("#next").click(function(){
        getData(count(true));
    });
    $("#prev").click(function(){
        getData(count(false));
    });
});

function getData(start) {
    $.get("http://localhost:8080/webService/api/velocitiespagination?start=" + start, function(velocities){
        createTable(velocities);
        $("#prev").prop("disabled", start === 0);
        $("#next").prop("disabled", velocities.length === 0);
    });
}

function createTable(velocities) {
    var txt="<tbody><tr><th>Tier</th><th>Maximalgeschwindigkeit</th></tr>"
    $.each( velocities, function( key, velocity ) {
        txt+="<tr><td>"+velocity.animal+"</td><td>"+velocity.velocity+"</td></tr>";
    });
    txt+="</tbody>"
    $("#myTable").html(txt);
}
```

Document ready, stellt sicher, dass erst das gesamte HTML geladen ist.

Closure, um den Counter hoch oder runter zu zählen

Click-Event dem Button zuordnen und in der anonymen Funktion den AJAX-Get-Aufruf ($.get(…)) definieren.

Die anonyme Funktion bekommt das Ergebnis des AJAX-Aufrufs, den Array velocities mit den velocity-Objekten, übergeben. In der Funktion wird dann createTable aufgerufen, welche dynamisch die Tabelle auf Basis der velocity-Objekte erzeugt.

Kommt noch was oder ist man am Ende der Pagination angekommen?

Zugriff auf die Properties des velocity-Objekts